© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

3 Relationale Datenbanksysteme

3.1 Hinführung

In der Unternehmenspraxis werden im Umfeld der betriebswirtschaftlichen Kernaufgaben Standard-Softwaresysteme eingesetzt, die auf einem Datenbanksystem als Plattform für die Datenverwaltung basieren. Insbesondere bei ERP-Systemen (▶ Kap. 1.1.6) finden sich häufig Datenbanksysteme, die das **relationale Datenmodell** als Grundlage haben. Man spricht hier von *relationalen Datenbanksystemen*.

Unter einem **relationalen Datenbanksystem** versteht man ein Datenbanksystem, welches die Eigenschaften des relationalen Datenmodells unterstützt. Die Ausrichtung eines Datenbanksystems nach dem relationalen Datenmodell bedeutet, dass das Denkmuster für die Organisation der Daten nach den Grundelementen des relationalen Datenmodells ausgerichtet ist.

Relationale Datenbanksysteme werden häufig durch das Akronym *RDBMS* beschrieben. *RDBMS* steht für **R**elationales **D**aten**b**ank-**M**anagement**s**ystem. Bekannte Beispiele relationaler Datenbanksysteme im betrieblichen Umfeld sind Microsoft ACCESS, Microsoft SQL Server, ORACLE sowie MySQL.

Die Definition der Struktur und Eigenschaften des relationalen Datenmodells geht auf E. F. Codd (1970) zurück. Das relationale Datenmodell besitzt wichtige Eigenschaften, die für die Speicherung und den logischen Umgang maßgebend sind. Die Struktur des relationalen Datenmodells wird auch **Relationenschema** genannt.

3.2 Grundstruktur und Eigenschaften der Tabelle

3.2.1 Aufbau einer Tabelle

Das zentrale Element des relationalen Datenmodells ist im mathematischen Sinne eine **Relation**. Anstelle des mathematischen Begriffs einer Relation wird sehr häufig von einer **Tabelle** gesprochen. Oftmals ist das Verständnis von einer Tabelle in vielen Anwendungsbereichen sehr unterschiedlich. An die Tabelle im relationalen Sinne bzw. ihre Eigenschaften werden bestimmte Anforderungen gestellt, die es jeweils zu beachten gilt.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Folgende Eigenschaften sind dem relationalen Datenmodell auferlegt:

- Als **Strukturierungselement** steht ausschließlich die Tabelle zur Verfügung, d. h. alle Daten und ihre sachlogischen Zusammenhänge müssen in Tabellen untergebracht werden.
- Die zu beschreibenden Objektmengen, beispielsweise *Kunden*, *Artikel*, *Lieferanten* und *Rechnungen*, werden im logischen Schema in Form von Relationen realisiert.
- Beziehungen zwischen den Objektmengen, beispielsweise *Rechnung gehört zu Kunde*, werden nicht über physische Verbindungen der Tabellen, sondern über inhaltliche Wertbezüge in den Tabellen hergestellt.

Die Grundstruktur und die wesentlichen Elemente einer Tabelle zeigt Abbildung 3.1.

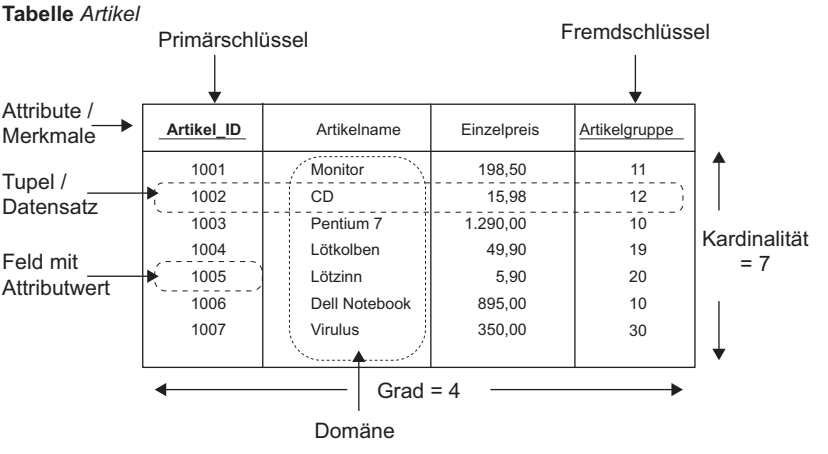


Abb. 3.1: Grundstruktur einer Tabelle

Jede Tabelle besteht aus Zeilen und Spalten. Die **Struktur** (statischer Aspekt) einer Tabelle wird durch die Spalten, der **Inhalt** (dynamischer Aspekt) einer Tabelle durch die Zeilen bestimmt. In Abbildung 3.1 hat die Tabelle *Artikel* genau 4 Spalten. Die Anzahl der Spalten wird durch den **Grad** der Tabelle, hier also Grad = 4, definiert. Die **Kardinalität**, hier 7, gibt die Zahl der zum aktuellen Zeitpunkt in der Tabelle enthaltenen Datensätze an.

Zeilen enthalten zusammengehörige Daten, die durch Spalten in Attribute bzw. Merkmale unterteilt und kategorisiert werden. Eine Tabelle wird in diesem Zusammenhang auch als **Relation** und eine Zeile als **Tu-**

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

pel bezeichnet. Alle Daten einer Zeile entsprechen genau einem Datensatz und repräsentieren eine sogenannte **Entität** der Datenbank.

Die Spalten der Tabelle definieren die Eigenschaften der Entitäten in der jeweiligen Zeile und stellen die einzelnen **Attributwerte** oder Merkmalswerte eines Datensatzes dar. Die Bezeichnung der Tabellenspalte repräsentiert den **Attributnamen**.

Da eine Tabellenspalte homogen ist, also in jeder Zelle der Spalte gleichartige Daten enthält, z. B. ganzzahlige Werte, können den Spalten bzw. den einzelnen Feldern einer Spalte feste **Datentypen** zugewiesen werden. Dies erleichtert dem Datenbankmanagementsystem den Umgang mit den Werten eines Feldes. Der Wertebereich der Feldwerte einer Spalte nennt sich auch **Domäne**. Seitens des DBMS ist die Integritätsbedingung zu prüfen, ob die Attributwerte eines Datensatzes auch der jeweils vorgegebenen Domäne entsprechen.

3.2.2 Zentrale Eigenschaften

Eine Tabelle im relationalen Sinne muss bestimmte Anforderungen an Eigenschaften erfüllen. Die zentralen Eigenschaften einer Tabelle sind folgendermaßen festgelegt:

- Jedes Feld einer Tabelle ist **atomar**, d. h. innerhalb eines Feldes dürfen keine logischen Unterstrukturen vorhanden sein.
  Beispiel: Das Feld *Anschrift* besteht inhaltlich aus einer *PLZ* und einem *Wohnort*. *PLZ* und *Wohnort* müssen als eigenständige Spalten in der Tabelle aufgebaut werden und dürfen nicht zusammen in einer Spalte wie *Anschrift* geführt werden. Die Eigenschaft atomarer Felder entspricht der Forderung nach der 1. Normalform (▶ Kap. 3.7.3).
- Die Reihenfolge in der Anordnung der Datensätze (Tupel) innerhalb einer Tabelle ist für die Anwendungen nicht relevant. Ein Zugriff nach der Ordnung der Datensätze ist damit nicht möglich, es kann beispielsweise in einer Datenabfrage nicht der »Datensatz an dritter Stelle« ausgewählt werden.
- Die Anordnung der Spalten bzw. Attribute innerhalb einer Tabelle hat hinsichtlich der Reihenfolge keine Bedeutung. Daraus folgt, dass Spalten immer über deren Bezeichnung referenziert werden und nicht über deren Reihung.
- Die **Objektintegrität** verlangt, dass jeder Datensatz in einer Tabelle eindeutig identifizierbar sein muss. Dafür wird mindestens ein Attribut des Datensatzes definiert, das den sogenannten **Primärschlüssel** (engl.: *Primary key*) darstellt. Der Wert dieses Feldes muss in der ganzen Ta-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

belle eindeutig sein. Darüber hinaus wird die Existenz eines Primärschlüsselwertes für alle Datensätze gefordert. Die Forderung nach der Objektintegrität wird auch als **1. Integritätsregel** bezeichnet.
- Neben dem Primärschlüssel können Tabellen einen sogenannten **Fremdschlüssel** enthalten. Der Fremdschlüssel muss zu einem Primärschlüssel einer anderen Tabelle oder der gleichen Tabelle korrespondieren. Jeder verwendete Fremdschlüsselwert muss daher auf den Wert eines Primärschlüssels verweisen. Diese Forderung wird Beziehungsintegrität bzw. **referentielle Integrität** genannt und als **2. Integritätsregel** bezeichnet.

Durch den Verweis auf einen anderen Datensatz, eigentlich auf den Primärschlüssel eines anderen Datensatzes, lassen sich Datensätze und somit in der Logik auch ganze Tabellen miteinander verknüpfen und in Beziehung zueinander setzen.

3.3 Schlüsselbegriffe im Datenmodell

3.3.1 Klassifikation der Schlüsselbegriffe

In der relationalen Datenbanktechnologie spielen verschiedene Schlüsselbegriffe eine wichtige Rolle. Nach ihrer grundsätzlichen Bedeutung können zunächst unterschieden werden:

- **Primärschlüssel** (engl.: *Primary key*): Sie dienen zur Datenidentifikation.
- **Sekundärschlüssel**: Sie dienen zur Unterstützung einer performanten Datenverarbeitung, insbesondere bei der Suche in großen Datenbeständen.
- **Fremdschlüssel** (engl.: *Foreign key*): Sie dienen der Herstellung logischer Beziehungen zwischen den Datensätzen verschiedener Tabellen.

Weitere Schlüsselbegriffe ordnen sich in diese Klassifikation ein. Beispielsweise dienen sogenannte **Schlüsselkandidaten** als potenzielle Primärschlüssel. Die Schlüsselbegriffe Primär- und Fremdschlüssel spielen in der strukturellen Gestaltung von Tabellen sowie in der Normalisierung (▶ Kap. 3.7) hinsichtlich der Feststellung bestimmter Eigenschaften eine wichtige Rolle.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

3.3.2 Primärschlüssel

Die relationale Datenbanktheorie basiert auf dem Grundsatz, dass jeder Datensatz (Tupel) einer Tabelle und damit jede Entität der Datenbank identifizierbar sein muss. Das Datenbanksystem muss stets in der Lage sein, den Zugriff auf einen Datensatz über einen identifizierenden Wert sicherstellen zu können. Dieser Wert wird durch die Einführung eines Primärschlüssels gewährleistet und organisiert. Der **Primärschlüssel** hat damit eine **Identifikationseigenschaft** für die Datensätze und garantiert damit die Möglichkeit, jeden Datensatz in der Tabelle selektieren zu können.

Ein Primärschlüssel ist mit folgenden **Eigenschaften** ausgestattet:

- Ein Primärschlüssel besteht aus einem Attribut (Spalte) oder setzt sich aus einer minimalen Kombination von Attributen der Tabelle zusammen.
- Die Werte des Primärschlüssels einer Tabelle müssen **eindeutig** sein, d. h. kein Wert darf mehrfach auftreten.
- Existenz von Primärschlüsselwerten: Für jeden Datensatz in der Tabelle muss ein Primärschlüsselwert existieren, es sind also keine NULL-Werte[5] erlaubt.

Die Festlegung eines Primärschlüssels erfolgt durch Auswahl aus den natürlichen Attributen der Tabelle, z. B. *Name* und *Geburtstag* eines *Kunden* oder durch ein künstlich eingeführtes Attribut, z. B. *Kundennummer*. Man spricht dann von einem **künstlichen Primärschlüssel** oder einem sogenannten **Surrogatschlüssel** (Ersatzschlüssel). Ein künstlicher Primärschlüssel wird meist durch eine numerische Spalte dargestellt und eröffnet in der Handhabung bestimmte Vorteile. Die Einführung eines künstlichen Primärschlüssels sollte erst dann stattfinden, wenn eine natürliche Auswahl von Attributen der Tabelle die Identifizierungseigenschaft für die Datensätze erfüllen kann.

Hinsichtlich der Bestimmung eines Primärschlüssels und auch nachfolgend in der Behandlung der Normalisierungsregeln bzw. Normalformen, spielen weitere Schlüsselbegriffe eine wichtige Rolle. Ein **Schlüsselkandidat** (engl. *Candidate key*) ist ein Attribut oder eine minimale

5 Im Datenbanksprachgebrauch steht der sogenannte NULL-Wert für den Wert »undefiniert« bei leeren Feldern in Tabellen. Häufig sind nicht alle Daten in einer Tabelle bekannt und eingetragen; dadurch entstehen leere Felder, die mit dem NULL-Wert belegt werden.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Attributkombination, welche in der Tabelle als Identifikationsschlüssel verwendet werden kann. Aus den verfügbaren Schlüsselkandidaten wird schließlich einer zum Primärschlüssel ausgewählt. Von einem **Schlüsselattribut** spricht man, wenn dieses Attribut in mindestens einem Schlüsselkandidaten der Tabelle vorkommt. Die Menge der **Nichtschlüsselattribute** der Tabelle enthält alle Attribute, die in keinem Schlüsselkandidaten vorkommen.

3.3.3 Sekundärschlüssel

Sekundärschlüssel sind interne Datenstrukturen, die mit einem Primärschlüssel in einer Tabelle keine nähere Verknüpfung haben. Diese Art von Schlüsselbegrifflichkeit dient ausschließlich der Optimierung von Datenbankoperationen, insbesondere von Suchoperationen.

Beispiel:
In einer Telefonbuch-Datenbank werden *Telefonnummer*, *Name* und *Anschrift* der Besitzer verwaltet. Das Attribut *Telefonnummer* ist Primärschlüssel. Seitens der Datenbanknutzer erfolgen viele Anfragen nach der *Telefonnummer* eines bestimmten Telefonbesitzers durch Angabe des *Namens*. Die Suche in der Datenbank nach einem bestimmten Datensatz erfolgt hier typischerweise über den *Namen*, also nicht unter Verwendung des Primärschlüsselwertes. Um die Suche effizient in der Datenbank zu organisieren, werden interne Datenstrukturen angelegt, die mit möglichst wenigen Vergleichsoperationen an den richtigen Datensatz und damit an die gewünschte *Telefonnummer* führen.

Die angesprochene Datenstruktur nennt sich **Tabellenindex**. Ein Index ist also eine spezifische Datenstruktur, welche die Suche von Daten über ein bestimmtes Merkmal unterstützt. Ein Index ordnet, anschaulich beschrieben, die Einträge einer vorgegebenen Tabellenspalte in einer baumartigen Datenstruktur an.

In den Indexstrukturen gibt es für Datenbanksysteme eine Vielfalt von Varianten, die entsprechend den Bedürfnissen einer Performance ausgerichtet werden. Das Anlegen eines Index wird nicht automatisch bei jeder Tabelle für jede Spalte durchgeführt, sondern nur für gewünschte Spalten innerhalb von ausgewählten Tabellen.

Die Anlage und das Verwalten einer Indexstruktur nimmt zusätzlich Rechenzeit in Anspruch. Beispielsweise muss bei jedem neu eingefügten Datensatz in einer Tabelle die zugrundeliegende Baumstruktur neu

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

organisiert werden. Deshalb sollte man nur wirklich notwendige Indizes aufbauen.

Es gibt eindeutige Indizes (jeder Attributwert ist einmalig) oder mehrdeutige Indizes (mehrere Spaltenwerte können den gleichen Wert aufweisen). Primärschlüsselattribute werden in den Tabellen implizit mit einer eindeutigen Indexstruktur hinterlegt. Die Suche nach Datensätzen über den Primärschlüsselwert wird also automatisch durch Indizes unterstützt und ist daher performant. Numerische Schlüsselattribute sind sehr effizient hinsichtlich der Verwaltung in Indexstrukturen.

3.4 Entwurf von Tabellen

Für den Entwurf einer logischen Datenstruktur als Grundlage für das spätere Datenbanksystem sind verschiedene Vorgehensweisen denkbar. Eine Datenstruktur muss so konzipiert sein, dass sie in der Lage ist, die im zugrundeliegenden Problemumfeld relevanten Daten speichern und für die Anwendungen der Datenbank vollständig und korrekt zur Verfügung stellen zu können.

Bei der Erstellung eines relationalen Datenmodelles, also eines Tabellenschemas, kann die Datenwelt der Problemstellung in direkter Form in eine oder mehrere Tabellen abgebildet werden. Alternativ wird systematisch über einen konzeptionellen Entwurf zunächst ein konzeptionelles Datenmodell als fachliches Abbild der Datenwelt der Problemstellung angefertigt, welches dann in einem nachfolgenden Schritt in das Tabellenschema transferiert wird. Diese beiden Vorgehensweisen sollen nachfolgend beschrieben und im Ergebnis bewertet werden.

3.4.1 Direkter Entwurf von Tabellen

Ein möglicher Entwurfsvorgang besteht im direkten Abbild der Daten in ein Tabellenschema. Hier erfolgt die Abbildung der realen Datenwelt direkt in eine einzige, große Tabelle. Die relevanten Daten der Problemsituation werden in Attributen bzw. Spalten der Tabelle umgesetzt.

Beispiel:
Für die Verwaltung der Mitarbeiterdaten im Unternehmen soll ein Datenbanksystem aufgebaut werden. Hierzu wird folgende Tabelle entworfen und verwendet:

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Tabelle - Mitarbeiter

Mitarbeiter	Straße	PLZ	Ort	Telefon	Tel-App	Abteilung
HM-0188	Hauptstraße 45	80011	München	089/1732	2013	A-1021
LK-0990	Wechselgasse 3	60005	Frankfurt	069/1623	865	A-0792
AK-0589	Ostparkstraße 18	60005	Frankfurt	069/7123	182	A-1827
DS-0191	Sierichstraße 12	20100	Hamburg	040/4324	213	B-271
MH-0590	Hauptstraße 45	80011	München	089/1732	981	E-91
KE-1189	Sierichstraße 12	20100	Hamburg	040/4324	192	A-1732
ML-1290	Sierichstraße 12	20100	Hamburg	040/4324	2813	B-16
KK-0888	Ostparkstraße 18	60005	Frankfurt	069/7123	173	C-163
WE-0589	Ostparkstraße 18	60005	Frankfurt	069/7123	81	B-1341

Abb. 3.2: Mitarbeiterdaten im Tabellenschema

Aus dem gegebenen Beispiel werden auch schnell die Probleme ersichtlich, die sich aus einem direkten Entwurf in der Abbildung einer großen Tabelle, meist auch als *Flat-File-Tabelle* bezeichnet, ergeben:

- **Datenredundanz**
  In der Tabelle werden ein und dieselben Daten zu einem gleichen Objekt mehrmals gespeichert. Hier spricht man von Datenredundanz. Beispielsweise sind die Niederlassungen mit *Straße*, *PLZ* und *Ort* bei jedem Mitarbeiter vollständig in der Tabelle zu wiederholen, also mehrfach zu speichern.
  Die Datenredundanz birgt die Gefahr für das Entstehen von inkonsistenten Daten in der Tabelle. Werden aufgrund von Änderungen nicht alle mehrfach vorkommenden Daten berücksichtigt, so entstehen Widersprüche in den Daten.
- **Speicheranomalien**
  Speicheranomalien treten dann auf, wenn aufgrund der vorgegebenen Tabellenstruktur die Unterbringung oder die Änderung von Informationen nicht möglich wird. Bestimmte Datenbankoperationen können also aufgrund ungünstiger Datenstrukturen nicht ausgeführt werden. Man spricht von einer **Einfüge-**, **Änderungs-** oder **Lösch-Anomalie**.

Die angesprochenen Probleme von Datenredundanz und Speicheranomalien führen zur Überlegung, die Tabellenstruktur nach bestimmten Eigenschaften auszurichten. Eine systematische und zielführende Vorgehensweise, erwünschte Eigenschaften in Tabellen festzustellen und beim Entwurf anzustreben, findet unter dem Begriff der **Normalisierung** (► Kap. 3.7) statt.

Normalisierung bedeutet die Erzeugung von Tabellenstrukturen mit erwünschten Qualitätseigenschaften wie beispielsweise einer Redundanzfreiheit. Die Normalisierung von Tabellen erlaubt eine eingehende

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Strukturanalyse von Tabellen hinsichtlich notwendiger Tabelleneigenschaften sowie durch Anwendung von **Normalisierungsschemata** eine optimale Strukturgestaltung. Hierbei werden Entscheidungen getroffen, welche Spalten eine Tabelle aufnehmen darf sowie in welcher Weise eine Zerlegung von Tabellen in neue Tabellen zu erfolgen hat.

Beispiel:
Am Beispiel der Tabelle für die Mitarbeiterverwaltung (▶ Abb. 3.2) sollen die vorhin beschriebenen Nachteile einer Flat-File-Tabelle aufgezeigt werden.

Die entworfene Tabelle soll für die Erfassung von Mitarbeiterdaten dienen. Zu jedem Mitarbeiter soll die Niederlassung und Abteilung, in der er arbeitet, gespeichert werden. Die Adresse ein und derselben Niederlassung muss in der Mitarbeiter-Tabelle mehrmals abgespeichert werden.

Tabelle - Mitarbeiter

Mitarbeiter	Straße	PLZ	Ort	Telefon	Tel-App	Abteilung
HM-0188	Hauptstraße 45	80011	München	089/1732	2013	A-1021
LK-0990	Wechselgasse 3	60005	Frankfurt	069/1623	865	A-0792
AK-0589	Ostparkstraße 18	60005	Frankfurt	069/7123	182	A-1827
DS-0191	Sierichstraße 12	20100	Hamburg	040/4324	213	B-271
MH-0590	Hauptstraße 45	80011	München	089/1732	981	E-91
KE-1189	Sierichstraße 12	20100	Hamburg	040/4324	192	A-1732
ML-1290	Sierichstraße 12	20100	Hamburg	040/4324	2813	B-16
KK-0888	Ostparkstraße 18	60005	Frankfurt	069/7123	173	C-163
WE-0589	Ostparkstraße 18	60005	Frankfurt	069/7123	81	B-1341

Redundante Daten: Niederlassung in Hamburg

Abb. 3.3: Datenredundanz

Aus der gegebenen Datenredundanz resultierende Probleme sind naheliegend:

- hoher Eingabeaufwand und erhöhter Speicherplatzbedarf sowie
- erhöhter Pflegeaufwand, weil beim Ändern der Anschrift einer Niederlassung innerhalb der Tabelle zunächst alle betroffenen Datensätze gesucht und dann vollständig abgeändert werden müssen.

Eine **Integritätsverletzung** in den Daten liegt vor, wenn mehrfach vorhandene, ursprünglich identische Daten, nach einer Änderung nicht mehr übereinstimmen. Im Rahmen der Datenpflege wurden Änderungen vorgenommen, die nicht an allen Stellen nachvollzogen wurden:

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Tabelle - Mitarbeiter

Mitarbeiter	Straße	PLZ	Ort	Telefon	Tel-App	Abteilung
HM-0188	Hauptstraße 45	80011	München	089/1732	2013	A-1021
LK-0990	Wechselgasse 3	60005	Frankfurt	069/1623	865	A-0792
AK-0589	Ostparkstraße 18	60005	Frankfurt	069/7123	182	A-1827
DS-0191	Sierichstraße 12	20100	Hamburg	040/4324	213	B-271
MH-0590	Hauptstraße 45	80011	München	089/1732	981	E-91
KE-1189	Sierichstraße 16	20100	Hamburg	040/1635	192	A-1732
ML-1290	Sierichstraße 16	20100	Hamburg	040/1635	2813	B-16
KK-0888	Ostparkstraße 18	60005	Frankfurt	069/7123	173	C-163
WE-0589	Ostparkstraße 18	60005	Frankfurt	069/7123	81	B-1341

Integritätsverletzung nach Datenpflege: Niederlassung in Hamburg

Abb. 3.4: Integritätsverletzung nach Änderung von Daten

Redundante Daten in einer Tabelle führen leicht zur Dateninkonsistenz und damit verbunden zu erhöhtem Pflegeaufwand und Fehlinterpretation von Daten.

Dateninkonsistenz bezeichnet widersprüchliche, unvollständige und unkorrekte Daten.

3.4.2 Speicheranomalien in Datenstrukturen

Bei einer direkten Modellierung der Datenwelt in ein relationales Datenmodell bzw. Tabellenschema treten häufig Flat-File-Tabellen auf. Die so gebildeten Datenstrukturen können **Speicheranomalien** bedingen, die auf eine redundante und damit inkonsistente Datenhaltung zurückzuführen sind.

Tabelle - Mitarbeiter

Mitarbeiter	Straße	PLZ	Ort	Telefon	Tel-App	Abteilung
HM-0188	Hauptstraße 45	80011	München	089/1732	2013	A-1021
LK-0990	Wechselgasse 3	60005	Frankfurt	069/1623	865	A-0792
AK-0589	Ostparkstraße 18	60005	Frankfurt	069/7123	182	A-1827
DS-0191	Sierichstraße 12	20100	Hamburg	040/4324	213	B-271
MH-0590	Hauptstraße 45	80011	München	089/1732	981	E-91
KE-1189	Sierichstraße 16	20100	Hamburg	040/1635	192	A-1732
ML-1290	Sierichstraße 16	20100	Hamburg	040/1635	2813	B-16
KK-0888	Ostparkstraße 18	60005	Frankfurt	069/7123	173	C-163
WE-0589	Ostparkstraße 18	60005	Frankfurt	069/7123	81	B-1341

Lösch-Anomalie: *wird Mitarbeiter LK-0990 gelöscht, so verschwindet auch die Niederlassung in der Wechselgasse 3 in Frankfurt ganz aus der Datenbank*

Abb. 3.5: Speicheranomalien in der Tabelle

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Aufgrund von Abhängigkeiten in den Elementen einer Flat-File-Tabelle, beispielsweise zwischen Mitarbeiter und Niederlassung, kann es bei Nichtexistenz eines Elementes zum **Datenverlust** des anderen Elementes kommen. Anhand der Abbildung 3.5 sollen die möglichen Ausprägungen einer Speicheranomalie, die Einfüge-, Lösch- und Änderungsanomalie, dargestellt werden.

Beispiel :
Lösch-Anomalie

- Der Mitarbeiter *LK-0990* verlässt das Unternehmen und soll daher aus der Datenbank gelöscht werden.
- **Problem**: der Mitarbeiter *LK-0990* ist der letzte Mitarbeiter der Niederlassung in der *Wechselgasse* in *Frankfurt*. Mit dem Löschen des Datensatzes *LK-0990* verschwindet somit auch die Information über die Niederlassung aus der Datenbank.

Einfüge-Anomalie

- Eine neue Niederlassung hat eröffnet und soll in die Datenbank aufgenommen werden. Für die Niederlassung sind die zugehörigen Mitarbeiter noch nicht bekannt.
- **Problem**: Die neue Niederlassung kann nicht in der Tabelle aufgenommen werden, weil kein Mitarbeiter vorhanden ist und damit auch die Daten der Niederlassung nicht gespeichert werden können.

Änderungs-Anomalie

- Der Mitarbeiter LK-0990 wechselt die Niederlassung.
- **Problem**: der Mitarbeiter *LK-0990* ist der letzte Mitarbeiter der Niederlassung in der *Wechselgasse* in *Frankfurt*. Bei der Änderung des Datensatzes *LK-0990* auf die neuen Niederlassungsdaten verschwindet somit auch die Information über die Niederlassung in der *Wechselgasse* aus der Datenbank.

Die Beispiele zeigen, dass Speicheranomalien die Konsistenz der in der Datenbank gespeicherten Daten gefährden.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

3.4.3 Konzeptioneller Entwurf

Alternativ zum direkten Entwurf von Tabellenstrukturen führt der Weg über eine Vorstufe des logischen Datenmodells, nämlich eines konzeptionellen Datenmodells. Ein **konzeptionelles Datenmodell** beschreibt die fachlich logische Gesamtsicht auf die relevanten Daten eines Problemumfeldes bzw. Unternehmens.

Der Entwurf von Tabellen erfolgt hier über die Transformation eines konzeptionellen Datenmodelles in ein logisches Tabellenschema. Für die Abbildung des konzeptionellen Datenmodelles in eine Menge von Tabellen als logisches Datenmodell gibt es zugehörige **Transformationsregeln**.

Das Ergebnis einer Tabellenstruktur, als systematische Transformation aus einem konzeptionellen Datenmodell, ist meist schon ausreichend normalisiert und erfüllt wesentliche Qualitätseigenschaften eines Datenmodelles.

Die konzeptionelle Datenmodellierung und die Transformation in ein Tabellenschema werden in Kapitel 4 ausführlich beschrieben.

3.5 Logische Verknüpfung von Tabellen

3.5.1 Wertgleichheit von Feldern

In der Realität sind sachlogische Zusammenhänge in den Daten allgegenwärtig. Beispielsweise müssen *Rechnungen* einzelnen *Kunden* zugeordnet werden und *Aufträge* beinhalten *Artikelpositionen*. Die Abbildung von sachlogischen Beziehungen zwischen den Datenobjekten muss im relationalen Schema ausschließlich über Tabelleninhalte erfolgen. Eine physikalische Verwaltung von Beziehungen zwischen Datensätzen verschiedener Tabellen ist im relationalen Datenbanksystem nicht möglich.

Im relationalen Schema erfolgt die Abbildung von logischen Beziehungen zwischen Tabellen anhand des Prinzips der **Wertgleichheit von Feldern.** Wertgleichheit bedeutet, dass über spezifische Spalten verschiedener Tabellen bei Wertübereinstimmungen ein logischer Zusammenhang gegeben ist. Die hierfür aufgebauten Schlüsselspalten werden als sogenannte **Fremdschlüsselspalten** verwendet.

In einer Datenabfrage kann diese künstlich hergestellte Wertgleichheit für die Wiederherstellung der sachlogischen Beziehungen in den Daten eingesetzt werden.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Beispiel:
Eine Datenbank soll die Mitarbeiter und die Abteilungen eines Unternehmens verwalten. Hierfür werden zwei Tabellen *Mitarbeiter* und *Abteilung* angelegt:

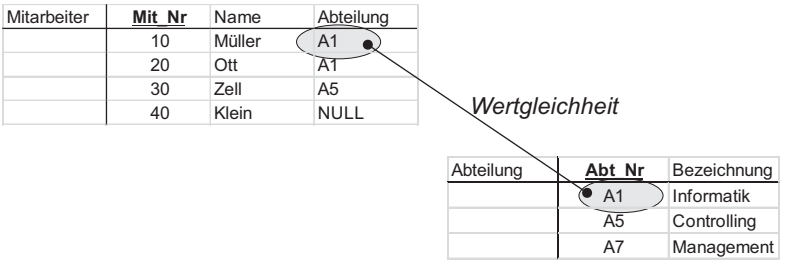


Abb. 3.6: Wertgleichheit in Tabellen

Die Zugehörigkeit eines Mitarbeiters zu einer Abteilung wird durch die **Wertgleichheit** in den beiden Spalten *Abteilung* (Tabelle *Mitarbeiter*) und *Abt_Nr* (Tabelle *Abteilung*) definiert. Gleiche Werte in den beiden Spalten repräsentieren eine sachlogische Beziehung zwischen den jeweiligen Datensätzen beider Tabellen. Der Mitarbeiter *Müller* trägt in der Spalte *Abteilung* den Attributwert *A1*. Der Wert *A1* in der Tabelle *Abteilung* identifiziert die Abteilung *Informatik*. Damit sind diese beiden Datensätze in der logischen Zusammengehörigkeit, d. h. *Müller* gehört zur Abteilung *Informatik*.

3.5.2 Fremdschlüssel

Um das Prinzip der Wertgleichheit in den Tabellen und damit im Datenbanksystem strukturell zu verankern, werden Fremdschlüsselspalten eingeführt. Fremdschlüssel bilden eine wichtige Funktion innerhalb eines relationalen Schemas.

Unter einem **Fremdschlüssel** versteht man Spalten einer Tabelle, die auf den Primärschlüssel einer anderen oder der gleichen Tabelle verweisen.

Damit das Prinzip der Wertgleichheit in Tabellen in einem Datenbanksystem in sich konsistent und anwendbar ist, müssen klare Regelungen getroffen und Eigenschaften eingehalten werden. Eine der wichtigsten **Ei-**

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

genschaften in diesem Zusammenhang ist die **Beziehungsintegrität** oder **referentielle Integrität**.

Die **Beziehungsintegrität** oder 2. Integritätsregel (▶ Abb. 3.7) umfasst folgenden Grundsatz: *Jeder Fremdschlüsselwert muss in der korrespondierenden Tabelle als Primärschlüsselwert existieren.*

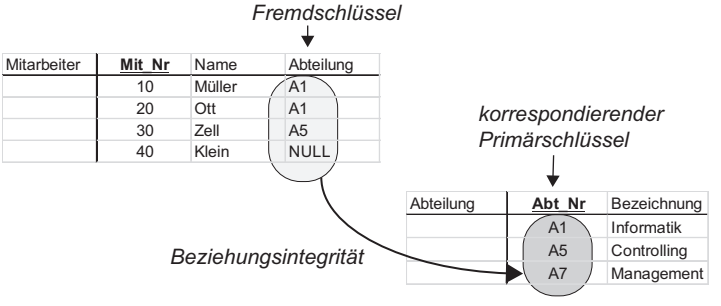


Abb. 3.7: Beziehungsintegrität in Tabellen

Jeder vorkommende Attributwert in der Fremdschlüsselspalte *Abteilung* in der Tabelle *Mitarbeiter* muss in der korrespondierenden Tabelle *Abteilung* als Wert in der Primärschlüsselspalte *Abt_Nr* enthalten sein.

Über die Beziehungsintegrität hinaus gelten noch weitere **Eigenschaften**, die einem Fremdschlüssel zugeordnet sind:

- Im Fremdschlüssel dürfen sogenannte NULL-Werte auftreten; für sie gilt die referentielle Integrität nicht (▶ Abb. 3.7: NULL-Wert in Spalte *Abteilung*).
- Die Werte im Fremdschlüssel müssen nicht eindeutig sein und können sich wiederholen.
- Besteht der korrespondierende Primärschlüssel aus mehreren Spalten, muss der Fremdschlüssel ebenfalls die gleiche Kombination an Spalten aufweisen.
- Der Spaltenname eines Fremdschlüssels darf frei gewählt werden. Die Namen der Spalten beim Fremdschlüssel und beim korrespondierenden Primärschlüssel müssen nicht übereinstimmend sein.
- Eine Tabelle kann mehrere Fremdschlüssel gleichzeitig zu unterschiedlichen und sogar zur eigenen Tabelle aufnehmen.
- Ein Fremdschlüssel, der auf den Primärschlüssel der eigenen Tabelle verweist, definiert eine **rekursive Beziehung**.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Die Einhaltung der Eigenschaften eines Fremdschlüssels sollte durch das jeweilige Datenbanksystem geprüft und überwacht werden. Hierfür bieten die Datenbanksysteme in aller Regel entsprechende Mechanismen zur Unterstützung der referentiellen Integrität an. Bei der Erstellung einer Tabellenstruktur, also beim physischen Entwurf der Tabellen, werden die Mechanismen auf die Fremd- und Primärschlüsselspalten zur Prüfung der referentiellen Integrität eingerichtet. Der funktionale Umfang für die Einhaltung und Prüfung der Datenintegrität ist in den Datenbanksystemen unterschiedlich ausgeprägt.

3.6 Tabellenentwurf bei Fremdschlüsselaufnahme

Für den Einsatz von Fremdschlüsseln zur Realisierung von logischen Beziehungen zwischen den Tabellen müssen bestimmte **Gesetzmäßigkeiten** beachtet werden. Die Entscheidung über die Unterbringung von Fremdschlüsselspalten ist von der **Beziehungskomplexität** abhängig. Daher muss ein Blick auf die gegebenen Komplexitätsgrade einer Beziehung geworfen werden.

3.6.1 Komplexitätsgrade in einer logischen Beziehung

Die Abbildung von sachlogischen Beziehungen zwischen den Datenobjekten im relationalen Tabellenschema ist nicht nur davon abhängig, welche Objekte aus welchen Tabellen zueinander in Beziehung treten, sondern auch davon, in welcher Vielfachheit die Beziehungen zwischen den Datensätzen der Tabellen auftreten dürfen oder müssen. Die möglichen Vielfachheiten in der Existenz von Beziehungen zwischen den Datensätzen der beteiligten Tabellen werden durch sogenannte Komplexitätsgrade definiert.

Der **Komplexitätsgrad** einer Tabelle *T* im Hinblick auf eine Beziehung *B* beschreibt, wie oft jedes Tupel der Tabelle *T* an der logischen Beziehung *B* beteiligt sein muss bzw. darf.

Für die Anwendung der Komplexitätsgrade in der Festlegung der Eigenschaften einer logischen Beziehung wird eine spezifische Notation verwendet. Die in diesem Buch angewandte Notation orientiert sich an die Definition von Komplexitätsgraden nach Schlageter/Stucky (1983). Diese

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Notation wird auch in der konzeptionellen Datenmodellierung (▶ Kap. 4) eingesetzt, dort aber detaillierter in der Definition behandelt.

Mit folgender Notation und inhaltlicher Bedeutung sind die Komplexitätsgrade wie folgt definiert:

Symbolik	Komplexitäts-grad	Bedeutung
`1`	`1`	jedes Tupel ist **genau einmal** an der Beziehung beteiligt
`c`	`{0,1}`	jedes Tupel ist **höchstens einmal** (**c**hoice) an der Beziehung beteiligt
`m`	`[1, ∞)`	jedes Tupel ist **mindestens einmal** an der Beziehung beteiligt (**m**ultiple)
`mc`	`[0, ∞)`	jedes Tupel kann **beliebig oft** (**m**ultiple **c**hoice) an der Beziehung beteiligt sein

Beispiel:
Die beiden Tabellen *Mitarbeiter* und *Abteilung* stehen in einer logischen Beziehung zueinander. Die Beziehung *Mitarbeiter gehört zur Abteilung* wird in dem Spaltenpaar (*Abteilung, Abt_Nr*) über das Prinzip der Wertgleichheit in Tabellen zum Ausdruck gebracht.

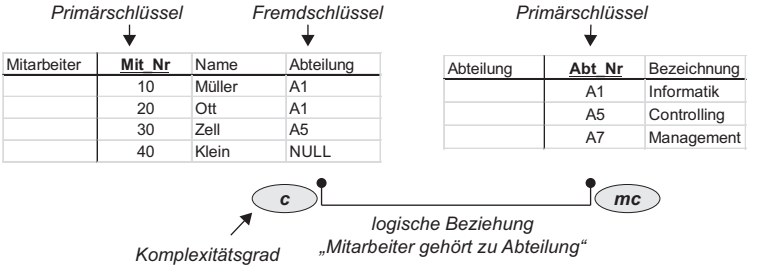


Abb. 3.8: Beispiel einer logischen Beziehung

Im vorliegenden Beispiel ist der Komplexitätsgrad der *Mitarbeiter* an der Beziehung gleich *c*. Damit kann jeder *Mitarbeiter* höchstens einmal eine Beziehung aufnehmen, d. h. jeder *Mitarbeiter* kann höchstens einer *Abteilung* zugeordnet sein. Mitarbeiter 10, 20 und 30 sind einer Abteilung, Mitarbeiter 40 keiner Abteilung zugeordnet.

Der Komplexitätsgrad der Abteilung an der Beziehung ist gleich *mc*. Damit kann jede *Abteilung* beliebig oft eine Beziehung aufnehmen,

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

d. h. jede Abteilung kann beliebig vielen Mitarbeitern zugeordnet sein. Abteilung *A1* hat zwei *Mitarbeiter*, *A5* hat einen *Mitarbeiter* und *A7* hat momentan keinen *Mitarbeiter* zugeordnet.

3.6.2 Regeln für die Fremdschlüsselaufnahme

Bei der Aufteilung von Daten auf verschiedene Tabellen und bei der Definition von Verknüpfungen übernimmt die Komplexität einer Beziehung zwischen Tabellen eine wichtige Rolle. Die Entscheidung darüber, welche Fremdschlüssel in welcher Tabelle positioniert werden dürfen, ist vom Komplexitätsgrad der Beziehung abhängig.

Die nachfolgenden Regeln unterscheiden dabei zwischen hierarchischen, konditionalen und netzwerkförmigen Beziehungen:

Regel für hierarchische Beziehungen

Eine **hierarchische Beziehung** liegt dann vor, wenn der Komplexitätsgrad bei (mindestens) einer Tabelle den Wert **1** hat.

Der Fremdschlüssel wird in diejenige Tabelle aufgenommen, die den Komplexitätsgrad **1** bei sich trägt.

Beispiel:
Jede *Rechnung* bezieht sich auf genau einen *Kunden*. Der Komplexitätsgrad der Tabelle *Rechnung* an der Beziehung ist vom Wert **1**, also hierarchisch. Damit wird in der Tabelle *Rechnung* der Fremdschlüssel *Kunde_Nr* aufgenommen, um die Beziehung zu unterhalten.

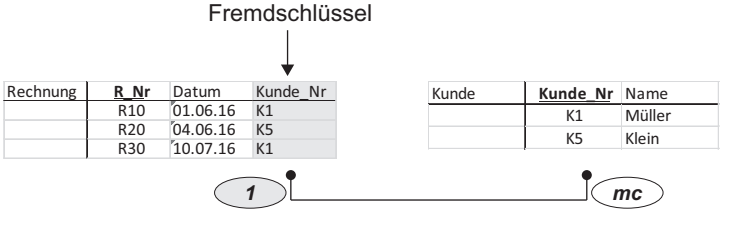


Abb. 3.9: Fremdschlüsselaufnahme bei hierarchischer Beziehung

Die alternative Lösung, in der Tabelle *Kunde* einen Fremdschlüssel *R_Nr* für die Beziehung aufzunehmen, wäre nicht machbar. Beispielsweise hat Kunde *K1* zwei Rechnungen, *R10* und *R30*, vorliegen. Eine

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Liste von Werten kann in einem Tabellenfeld nicht aufgenommen werden, weil dies gegen die atomare Eigenschaft eines Feldes verstößt.

Regel für konditionale Beziehungen

Eine **konditionale Beziehung** liegt dann vor, wenn der Komplexitätsgrad bei (mindestens) einer Tabelle den Wert **c** hat.

Der Fremdschlüssel wird in diejenige Tabelle aufgenommen, die den Komplexitätsgrad *c* bei sich trägt. Dieser Lösungsansatz setzt aber voraus, dass in der Fremdschlüsselspalte auch ein NULL-Wert stehen darf.

Beispiel:
Jeder *Mitarbeiter* kann einer *Abteilung* zugeordnet sein. Der Komplexitätsgrad der Tabelle *Mitarbeiter* an der Beziehung ist optional, d. h. vom Wert **c**. Die Aufnahme des Fremdschlüssels erfolgt daher in der Tabelle *Mitarbeiter*.

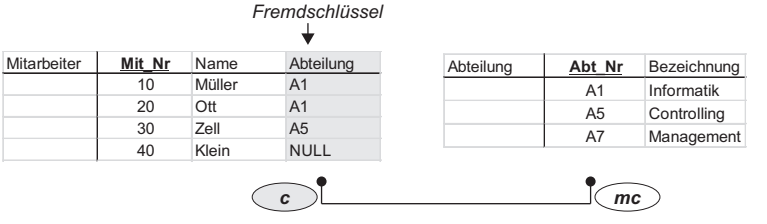


Abb. 3.10: Fremdschlüsselaufnahme bei konditionaler Beziehung

Regel für netzwerkförmige Beziehungen

Eine **netzwerkförmige Beziehung** liegt dann vor, wenn alle Komplexitätsgrade der beteiligten Tabellen vom Wert **m** oder **mc** sind (damit also weder eine hierarchische noch eine konditionale Beziehung gegeben ist).

In dieser Situation ist die Fremdschlüsselaufnahme in keine der bestehenden Tabellen möglich und es muss eine neue Tabelle geschaffen werden, die alleine zur Realisierung der Beziehung dient. Hier wird oft von Beziehungstabellen gesprochen, obwohl in relationalen Schemata alle Tabellen vom Datenbanksystem als gleichwertig gesehen werden.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Beispiel:
Jeder *Mitarbeiter* kann in verschiedenen *Projekten* beteiligt sein. Der Komplexitätsgrad der Tabelle *Mitarbeiter* an der Beziehung ist vom Wert **mc** und der Komplexitätsgrad der Tabelle *Projekt* vom Wert **m** (jedes Projekt muss mindestens einen Mitarbeiter besitzen). Es liegt eine netzwerkförmige Beziehung vor.

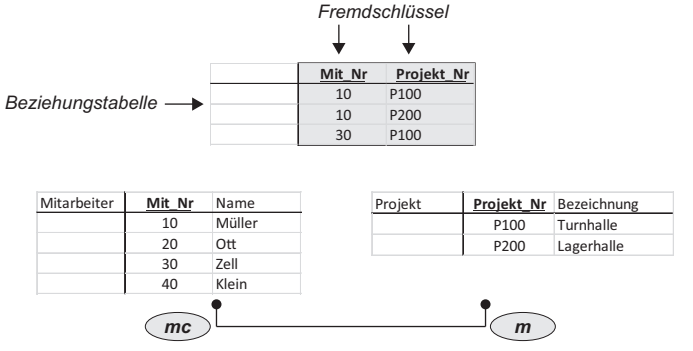


Abb. 3.11: Fremdschlüsselaufnahme bei netzwerkförmiger Beziehung

Für die Umsetzung der Beziehung zwischen den beiden Tabellen wird eine neue Tabelle eingerichtet, die sich in ihren Attributen alleine nur aus den Fremdschlüsselattributen der beteiligten Tabellen *Mitarbeiter* und *Projekt* zusammensetzt.

Der Primärschlüssel der neuen Beziehungstabelle setzt sich aus der Kombination beider Fremdschlüsselattribute zusammen.

3.7 Normalisierung

3.7.1 Motivation

Nach der Erstellung eines logischen Datenmodells müssen die Datenstrukturen hinsichtlich erwünschter Eigenschaften geprüft und auf potenzielle Probleme wie beispielsweise Speicheranomalien hin untersucht sowie bei Bedarf optimiert werden. Die Konsistenz und Integrität der Daten sind zentrale Eigenschaften in den Datenbanksystemen. In diesem Kontext spielt die Redundanzfreiheit von Daten eine zentrale Rolle. Das Ziel, eine hohe Qualität in den Daten eines Datenbanksystems zu erlangen, setzt die Möglichkeit einer Bewertung des zugrundeliegenden Datenmodelles, also der Struktur der Daten, voraus.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Mit der **Normalisierung** wurden im relationalen Datenmodell sogenannte **Normalformen** eingeführt, die nach bestimmten Regeln die Redundanzfreiheit in den Relationen bewerten. Die Normalisierung ist ein formal fundierter Prozess, der auf dem mathematischen Modell des Relationenbegriffs nach E. F. Codd aufbaut. Ohne die Theorie der Normalisierung ganz auszublenden, soll hier im Lehrbuch eine praxisnahe Anwendung der Normalisierungsregeln aufgezeigt werden.

Über die Bewertung der Redundanzfreiheit hinaus enthält die Normalisierung auch Schemata, die die **Herstellung normalisierter Tabellen** aufzeigen und damit einen konstruktiven Ansatz verfolgen. Um eine redundante Datenhaltung innerhalb eines relationalen Datenmodells und die dadurch möglicherweise auftretenden Inkonsistenzen und Anomalien zu vermeiden, werden also die Relationen systematisch bewertet und normalisiert.

Die Normalisierung verfolgt einen systematischen Prozess der strukturellen Zerlegung der Relationen, um Redundanzen zu eliminieren und die Datenstruktur möglichst effizient zu gestalten. Das Datenmodell wird dabei ohne Datenverlust schrittweise normalisiert und stufenweise in mehrere sogenannte Normalformen überführt.

In der Theorie gibt es mehrere Normalisierungsstufen, die aufeinander aufbauen. Insgesamt gibt es fünf Stufen verschiedener Normalformen. In der Praxis der Datenmodellierung wird eine Normalisierung von der ersten bis zur dritten Normalform als ausreichend erachtet, obwohl noch die weiteren Stufen der 4. und 5. Normalform sowie die Boyce-Codd'sche Normalform existieren.

Da bei der Normalisierung die Relationen immer weiter zerlegt werden, entstehen viele Tabellen, wobei im Sinne der Effizienz auf eine minimal notwendige Zahl von Tabellen hingearbeitet wird. Bereits bei einfachen SQL-Abfragen wird es dadurch notwendig, Daten aus mehreren Tabellen abzurufen. Dies kann bei sehr großen Datenbeständen eine hohe Rechenleistung erfordern und daher für die Performance eines Systems nachteilig wirken. Aus diesem Grund werden Normalisierungen auch teilweise wieder rückgängig gemacht, um dem konkurrierenden Ziel einer Performanceverbesserung gerecht zu werden. Hier spricht man von einer **Denormalisierung** der Relationen wie sie beispielsweise in den Datenmodellen eines Star-Schemas (▶ Kap. 5.5) im Data Warehouse-System vorzufinden ist.

Einem wohlgeordneten Entwicklungsprozess folgend, erfolgt die Erstellung des relationalen Datenmodells durch Transformation des konzeptionellen Datenmodells wie beispielsweise eines Entity-Relationship-Modells in das Tabellenschema. Die Qualität der Tabellenstruktur hinsichtlich der Erfüllung erwünschter Normalformen ist natürlich von der Einhaltung

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

bestimmter Regeln der Normalisierung im ER-Modell abhängig. Die Transformationsregeln in der Abbildung eines ER-Modells in das zugehörige Relationenschema (▶ Kap. 4.5) sind an der Erfüllung der dritten Normalform ausgerichtet. In der konzeptionellen Datenmodellierung ist es deshalb erforderlich, die Normalisierungsregeln einer redundanzfreien Datenstruktur zu befolgen, um die Qualität nachfolgender Datenmodelle zu unterstützen.

3.7.2 Funktionale Abhängigkeit

In den vorangegangenen Beispielen von Tabellen wurden immer wieder Probleme aufgezeigt, beispielsweise anhand von Speicheranomalien, die letztlich aufgrund von spezifischen Abhängigkeiten zwischen den Attributen in einer Tabelle erklärbar sind. Die Tabelle in Abbildung 3.5 zu den Speicheranomalien ist beispielgebend. In einer leicht modifizierten Form soll diese Tabelle näher analysiert werden.

Tabelle *Mitarbeiter*

PS

Mitarbeiter	Name	Straße	PLZ	Ort	Telefon	Tel-App	Abteilung
HM-0188	Müller	Hauptstraße 45	80011	München	089/1732	2013	A-1021
LK-0990	Klein	Wechselgasse 3	60005	Frankfurt	069/1623	865	A-0792
AK-0589	Fischer	Ostparkstraße 18	60005	Frankfurt	069/7123	182	A-1827
DS-0191	Mann	Sierichstraße 12	20100	Hamburg	040/4324	213	B-271
MH-0590	Sauber	Hauptstraße 45	80011	München	089/1732	981	E-91
KE-1189	Breitner	Sierichstraße 12	20100	Hamburg	040/4324	192	A-1732

Abb. 3.12: Abhängigkeiten in den Attributen

Die Spalten *Mitarbeiter*, *Name* und *Ort* sollen näher betrachtet werden. Die Spalte *Mitarbeiter* ist Primärschlüssel der Tabelle und damit identifizierend für die Datensätze. Ein Datensatz beschreibt also die zu jedem Mitarbeiter gespeicherten Eigenschaften. *Name* als Spalte beschreibt eine direkte Eigenschaft des jeweiligen Mitarbeiters. Die Spalte *Ort* beschreibt, in welcher Niederlassung der Mitarbeiter arbeitet. Diese Eigenschaft resultiert aus der logischen Beziehung zwischen den Mitarbeitern und den vorhandenen Niederlassungen. Damit ist die Spalte *Ort* als Eigenschaft von Mitarbeiter **indirekt** über die Beziehung gegeben, während die Spalte *Name* **direkt** eine Eigenschaft des Mitarbeiters als Objekt ist.

Die in den Speicheranomalien aufgetretenen Probleme dieser Tabelle resultieren aus der in der Tabelle vorkommenden Mehrfachspeicherung (Redundanz) von Niederlassungen (siehe Spalten *Straße*, *PLZ*, *Ort*, *Telefon*) sowie deren Existenzabhängigkeit zu Mitarbeitern. Die in der Spalte

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Ort gehaltenen Informationen sind also problematisch, während die Spalte *Name* nicht verursachend für Anomalien ist. Aus der näheren Betrachtung der **Abhängigkeiten zwischen diesen Spalten** lässt sich zusammenfassend herausstellen: Die Spalte *Mitarbeiter* ist identifizierend in dieser Tabelle. Die Spalte *Name* ist eine direkte Eigenschaft von *Mitarbeiter*, während die Spalte *Ort* nur indirekt zu *Mitarbeiter* als Eigenschaft gehört und durch eine logische Beziehung zustande kam.

Die der Normalisierung zugrundeliegende Theorie zielt auf eine Analyse der funktionalen Abhängigkeit zwischen den Attributen einer Tabelle bzw. einer Relation.

Eine **funktionale Abhängigkeit** ist folgendermaßen definiert: Gegeben sei eine Relation *R* mit Attributen *A* und *B*. In einer Relation *R* ist das Attribut *B* vom Attribut *A* **funktional abhängig**, falls zu jedem Wert des Attributs *A* genau ein Wert des Attributs *B* gehört.
In der formalen Schreibweise wird dies im Weiteren mit folgendem Ausdruck dargestellt:

A $\rightarrow$ B (Sprechweise: *A* impliziert *B* oder *B* ist funktional abhängig von *A*)

Eine funktionale Abhängigkeit von zwei Attributen (oder Attributmengen) entsprechend der vorangegangenen Definition liegt vor, wenn der Wert des Attributs *B* vom Wert des anderen Attributs A abhängig ist, also der Wert des Attributs A den Wert des Attributs B beeinflusst. Anders ausgedrückt, **funktional abhängige Attribute** sind Attribute der realen Welt, die sich **nicht unabhängig voneinander** ändern können.

Beispiel:
Die Relation *Bauteil* besteht aus den Attributen *Teil-Nr*, *Gewicht* und *Farbe*. Es liegen folgende Bauteile vor:

Teil-Nr	Gewicht	Farbe
10	Klein	Rot
20	Klein	Blau
30	Groß	Weiss
40	Mittel	Weiss

Gewicht und *Farbe* eines Bauteils sind als Attribute von der *Teil-Nr* **funktional abhängig**, es gilt also:

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Teil-Nr → Gewicht
Teil-Nr → Farbe

Für zwei Datensätze mit unterschiedlichen Werten in der Spalte *Gewicht*, ergeben sich immer unterschiedliche Werte in der Spalte *Teil-Nr*, siehe *Teil-Nr* 20 und 30. Umgekehrt aber ist zu beachten, dass *Teil-Nr* **nicht funktional abhängig** vom *Gewicht* des Bauteils ist, es gilt also nicht:

Gewicht → Teil-Nr

Dies lässt sich leicht begründen: Bei Vorgabe des Wertes *Gewicht* = *Klein* ist der Wert für *Teil-Nr* nicht eindeutig (kann 10 oder 20 sein). Für unterschiedliche Werte in Spalte *Teil-Nr*, etwa 10 und 20, ergeben sich keine unterschiedlichen Werte in *Gewicht*, damit können sie sich nicht unabhängig voneinander ändern.

Die Regeln der Normalisierung basieren auf der Betrachtung **verschiedener Ausprägungen einer funktionalen Abhängigkeit.** Aufgrund der Fokussierung auf die drei in der Praxis bekanntesten Normalformen sind im Weiteren folgende funktionale Abhängigkeiten von Bedeutung:

- **funktionale Abhängigkeit**
- **volle funktionale** Abhängigkeit
- **transitive** Abhängigkeit

3.7.3 Erste Normalform (1NF)

Bei der ersten Normalform (kurz **1NF**) sind in den Relationen nur **atomare Felder**, also atomare Attributwerte erlaubt. Atomare Attributwerte dürfen nicht mehrwertig (keine Listen, Wiederholungsgruppen) sein und bestehen ausschließlich aus nicht strukturierten Datentypen (Standarddatentypen wie Zeichenketten, Zahlen). Das heißt, der Wertebereich eines Attributs kann nicht weiter in sinnvolle Teilbereiche untergliedert werden. Ein Feldwert ist damit nicht weiter unterteilbar.

Erste Normalform (1NF): Eine Relation genügt der ersten Normalform, wenn alle Attribute atomar sind.

Liegt eine Relation in erster Normalform vor, so determiniert ein gegebener Wert des Primärschlüssels eindeutig das zugehörige Tupel und damit die Werte sämtlicher Attribute. Damit liegt eine **funktionale Abhängigkeit aller Attribute vom Primärschlüssel** vor. Die erste Normalform ist damit

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

gleichbedeutend mit der Aussage, dass die funktionale Abhängigkeit der Attribute vom Primärschlüssel gegeben sein muss.

Im relationalen Datenmodell spricht man erst dann von einer *Tabelle*, wenn die erste Normalform, also die Forderung nach atomaren Feldern und der Existenz eines Primärschlüssels, erfüllt sind.

Die Verletzung der ersten Normalform in einer vorgegebenen Tabelle bedingt eine Umgestaltung hin zu einer 1NF-Tabelle. Ist der Wertebereich eines Attributs nicht atomar, muss das Attribut in weitere eigenständige Attribute aufgeteilt bzw. der Datensatz in mehrere Datensätze zerlegt werden. Die Redundanzen, die bei der Aufteilung entstehen, werden durch die nachfolgenden Normalformen wieder eliminiert.

Für die systematische Neustrukturierung einer gegebenen Relation hinsichtlich einer angestrebten Normalform lassen sich Zerlegungsprinzipien finden, die als **Normalisierungsschemata** definiert werden.

Normalisierungsschema für die erste Normalform

Eine Nicht-1NF-Ausgangstabelle kann in die erste Normalform überführt werden, indem für jede Wiederholungsgruppe sowie für jedes mehrwertige Attribut entsprechende Relationen erzeugt werden.

Ein mögliches Zerlegungsprinzip ist in Abbildung 3.13 dargestellt.

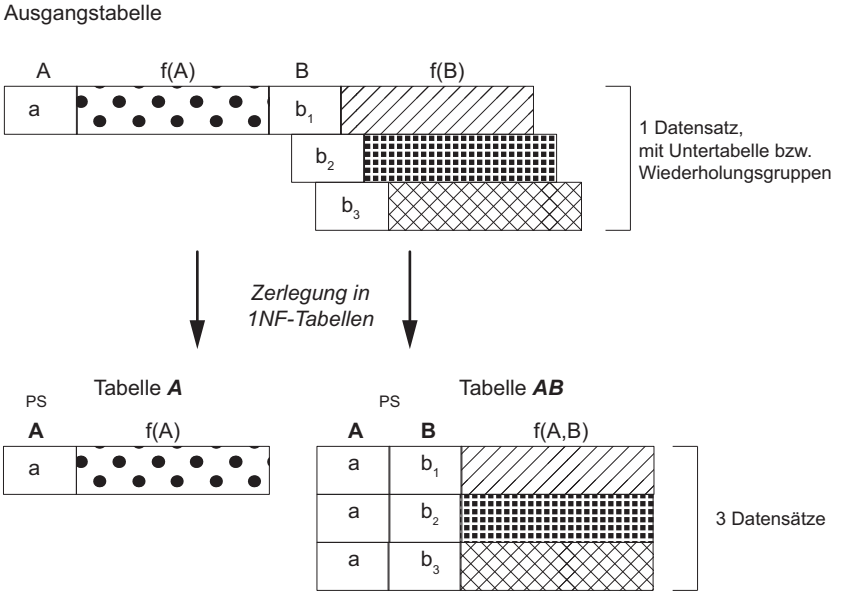


Abb. 3.13: Zerlegungsprinzip nach 1NF

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Beispiel:
Eine Nicht-1NF-Tabelle liegt in Form einer Ausgangstabelle für eine Projektverwaltung vor:

Ausgangstabelle *Projektverwaltung*

Projekt_Nr	Projekttitel	Budget	Mitarbeiter				
5	Büroautomation	900	Mit_Nr	Name	Beruf	Telefon	Arbeitszeit
			1	Schmidt	Informatiker	2334	40%
			2	Walter	Betriebswirt	1200	60%
3	Wertanalyse	200	Mit_Nr	Name	Beruf	Telefon	Arbeitszeit
			1	Schmidt	Informatiker	2334	30%
			3	Fischer	Controller	5400	70%

Feld von Spalte *Mitarbeiter* enthält
eine Untertabelle bzw. Wiederholungsgruppe

Abb. 3.14: Projektverwaltung

Die Verletzung der 1NF resultiert aus den Feldern in der Spalte *Mitarbeiter*, die jeweils für sich eigene Untertabellen der Projektmitarbeiter enthalten.

Eine Anwendung des Zerlegungsprinzips aus Abbildung 3.13 ergibt folgendes Ergebnis von 1NF-normalisierten Tabellen:

A f(A) B f(B)

Projekt_Nr	Projekttitel	Budget	Mitarbeiter				
5	Büroautomation	900	Mit_Nr	Name	Beruf	Telefon	Arbeitszeit
			1	Schmidt	Informatiker	2334	40%
			2	Walter	Betriebswirt	1200	60%
3	Wertanalyse	200	Mit_Nr	Name	Beruf	Telefon	Arbeitszeit
			1	Schmidt	Informatiker	2334	30%
			3	Fischer	Controller	5400	70%

Zerlegung nach Schema 1NF-Tabellen

PS
A f(A)

Projekt_Nr	Projekttitel	Budget
5	Büroautomation	900
3	Wertanalyse	200

Tabelle *Projekt*

PS
A B f(A,B)

Projekt_Nr	Mit_Nr	Name	Beruf	Telefon	Arbeitszeit
5	1	Schmidt	Informatiker	2334	40%
5	2	Walter	Betriebswirt	1200	60%
3	1	Schmidt	Informatiker	2334	30%
3	3	Fischer	Controller	5400	70%

Tabelle *Projektgruppe*

Abb. 3.15: Normalisierung der Projektverwaltung

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

3.7.4 Zweite Normalform (2NF)

Tabellen, die zunächst nur die erste Normalform erfüllen, sind oftmals durch redundante Datenhaltung geprägt und für Speicheranomalien anfällig. Für die weitere Normalisierung ist daher zu analysieren, welche Ursachen für diese potenziellen Probleme in den Abhängigkeiten der Attribute vorhanden sind.

Die **zweite Normalform** (2NF) beschäftigt sich mit der besonderen Form der **vollen funktionalen Abhängigkeit** von Attributen in einer Relation.

Beispiel:
Gegeben sei die Tabelle *Projektgruppe*, die als Ergebnis aus der ersten Zerlegung der Tabelle Projektverwaltung (▶ Abb. 3.15) entstanden ist und sich in der ersten Normalform befindet.

Tabelle *Projektgruppe*

PS

A B f(A,B)

Projekt_Nr	Mit_Nr	Name	Beruf	Telefon	Arbeitszeit
5	1	Schmidt	Informatiker	2334	40%
5	2	Walter	Betriebswirt	1200	60%
3	1	Schmidt	Informatiker	2334	30%
3	3	Fischer	Controller	5400	70%

Datenredundanz

Abb. 3.16: Problemanalyse zur zweiten Normalform

Die Tabelle Projektgruppe hat als Primärschlüssel die Attributkombination (*Projekt-Nr, Mit-Nr*), setzt sich also aus der Kombination der beiden Schlüsselattribute *Projekt-Nr* und *Mit-Nr* zusammen.

In den markierten Feldern ist erkennbar, dass Datenredundanz in der Tabelle vorliegt. Es gilt nun zu analysieren, worin die Ursachen für diese Datenredundanz liegen.

Auffallend ist in dieser Situation: Für das Attribut *Beruf* ist bereits das Primärschlüssel-Attribut *Mit-Nr* identifizierend, d. h. der *Beruf* eines Mitarbeiters kann bereits durch Angabe der *Mit-Nr* eindeutig ermittelt werden und benötigt keinerlei Informationen über zugehörige Projekte. Der Wert im PS-Attribut *Projekt-Nr* ist also für die funktionale

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Abhängigkeit nicht ausschlaggebend. Der identische Sachverhalt gilt auch für die Attribute *Name* und *Telefon*. Ein Attributwert von *Mit-Nr* darf in der Spalte mehrfach auftreten, weil nur die Kombination der PS-Attribute (*Projekt-Nr, Mit-Nr*) eindeutig sein muss. Mit jeder möglichen Wiederholung des Attributwerts von *Mit-Nr* wiederholt sich aufgrund der funktionalen Abhängigkeit von *Name*, *Beruf* und *Telefon* vom PS-Attribut *Mit-Nr* auch diese Werteliste und damit ist die Redundanz in dieser Tabelle erklärt.

Alle Nichtschlüsselattribute sind bei Vorliegen der ersten Normalform immer funktional abhängig von einem Primärschlüssel. Setzt sich ein Primärschlüssel aus mehreren Attributen zusammen und ein Nichtschlüsselattribut ist von allen Attributen des Schlüssels, nicht aber bereits von einem Teil des Schlüssels abhängig, spricht man von einer vollen funktionalen Abhängigkeit.

Die **volle funktionale Abhängigkeit** ist folgendermaßen definiert: Gegeben sei die Relation *R* mit den Primärschlüsselattributen S_1 und S_2 sowie der Attributmenge *M*. *A* sei ein Attribut aus *M*. In einer Relation $R(S_1,S_2, M)$ ist das Attribut *A* von den PS-Attributen S_1,S_2 **voll funktional abhängig**, wenn A von den zusammengesetzten Attributen (S_1,S_2) funktional abhängig ist, nicht aber bereits von einem einzelnen Attribut S_1 oder S_2.

Volle funktionale Abhängig bedeutet, dass jedes Nichtschlüsselattribut nur durch Angabe aller Attributwerte des Primärschlüssels identifiziert werden kann.

Zweite Normalform (2NF): Eine Relation ist in zweiter Normalform, falls sie in 1NF ist und jedes Nichtschlüsselattribut **voll funktional** vom Primärschlüssel **abhängig** ist.

Eine Relation in 1NF, deren Primärschlüssel aus genau einem Attribut besteht, ist offenbar immer auch zugleich in zweiter Normalform. Die volle funktionale Abhängigkeit kann nur bei einem mehrwertigen Primärschlüssel verletzt sein.

Normalisierungsschema für die zweite Normalform

Eine 1NF-Ausgangstabelle wird in 2NF überführt, indem für jede nicht voll funktional abhängige Attributgruppe eine eigene Relation herausgelöst wird.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Das Zerlegungsprinzip ist in Abbildung 3.17 dargestellt.

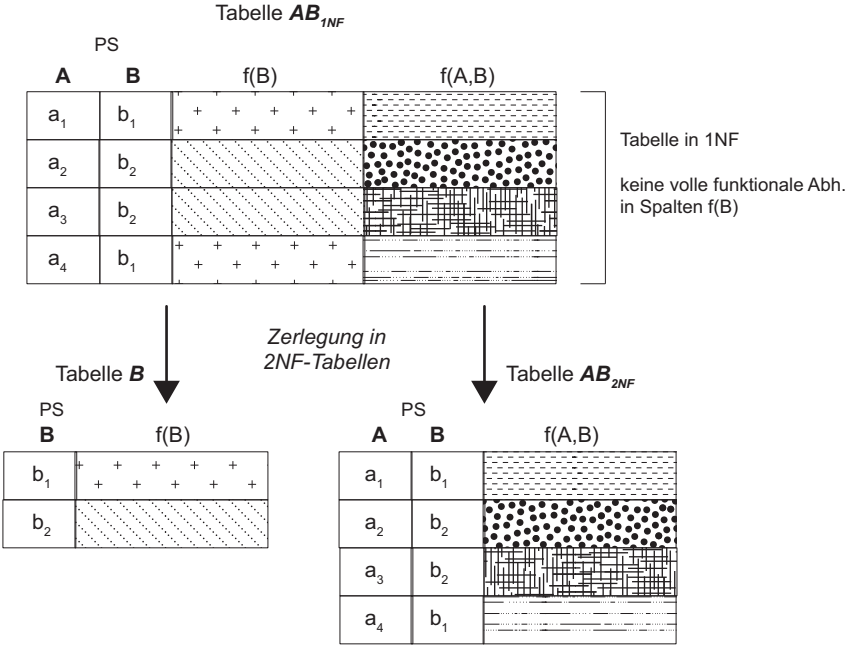


Abb. 3.17: Zerlegungsprinzip nach 2NF

Beispiel:
Gegeben sei die Tabelle *Projektgruppe*, die wie vorhin bereits aufgezeigt, redundante Datenstrukturen beinhaltet.

Für die Vorbereitung der Normalisierung müssen die vorhandenen Attribute in der Tabelle nach der Verletzung einer vollen funktionalen Abhängigkeit untersucht werden. Im Beispiel liegt folgende Situation zugrunde:

Tabelle *Projektgruppe*

PS		*nicht voll funktional abhängig*			***voll funktional*** *abhängig*
A	B	f(B)			f(A,B)
Projekt_Nr	**Mit_Nr**	Name	Beruf	Telefon	Arbeitszeit
5	1	Schmidt	Informatiker	2334	40%
5	2	Walter	Betriebswirt	1200	60%
3	1	Schmidt	Informatiker	2334	30%
3	3	Fischer	Controller	5400	70%

Abb. 3.18: Tabelle mit Verletzung der zweiten Normalform

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Die Attribute *Name, Beruf* und *Telefon* sind nicht voll funktional vom Primärschlüssel (Projekt-Nr, Mit-Nr) abhängig, weil diese Attribute bereits funktional abhängig vom PS-Attribut *Mit-Nr,* also vom einem Teil des Primärschlüssels, sind. Es gilt hier:

Mit-Nr → Name
Mit-Nr → Beruf
Mit-Nr → Telefon

Voll funktional abhängig ist lediglich das Attribut *Arbeitszeit,* also

(Projekt-Nr, Mit-Nr) → Arbeitszeit

Nach dem Zerlegungsschema für die Herstellung der zweiten Normalform müssen die zum Mitarbeiter gehörenden Attribute herausgelöst und in einer neuen Tabelle *Mitarbeiter* redundanzfrei eingebracht werden. Das Ergebnis ist in Abbildung 3.19 dargestellt.

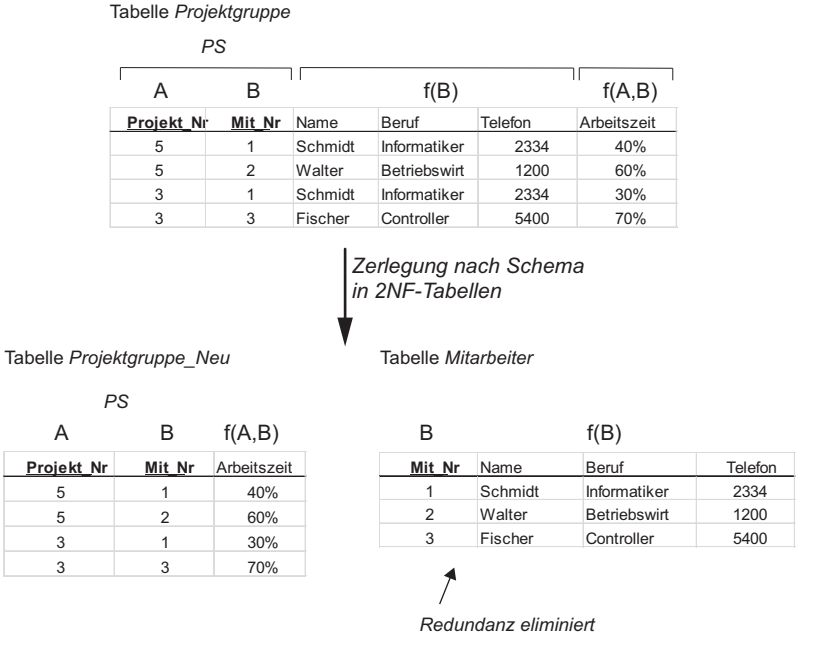


Abb. 3.19: Ergebnis der Zerlegung in zweite Normalform

Die erzeugten Tabellen *Projektgruppe_Neu* und *Mitarbeiter* erfüllen nun die zweite Normalform und es zeigen sich keine redundanten Werte mehr. Speicherungsanomalien sind in diesen konkreten Tabellen nicht mehr zu erwarten.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

3.7.5 Dritte Normalform (3NF)

Hinsichtlich der Analyse weiterer Normalformen stellt sich die Frage, ob es Tabellenkonstellationen gibt, die eine zweite Normalform erfüllen, aber dennoch von Datenredundanz und Speicheranomalien betroffen sind. Dazu betrachten wir ein modifiziertes Beispiel zu der Tabelle *Mitarbeiter* (▶ Abb. 3.20).

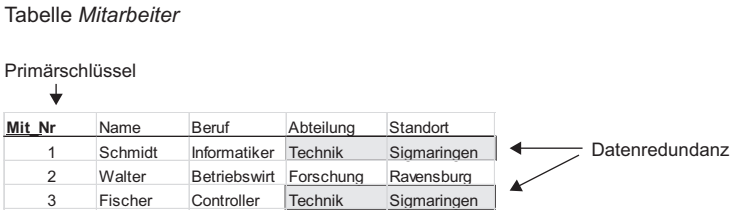


Abb. 3.20: Analyse der Tabelle Mitarbeiter

Die Relation *Mitarbeiter* erfüllt aufgrund des Primärschlüssels, der nur aus einem Attribut besteht, offensichtlich die zweite Normalform. In folgenden Situationen zeigt es sich, dass dennoch Redundanz und Speicheranomalien in der 2NF-Tabelle *Mitarbeiter* auftreten:

- in der Spalte *Standort* ist Redundanz erkennbar; die Abteilungsinformation wiederholt sich
- falls sich der Standort einer Abteilung ändert, sind mehrere Tupel der Relation zu ändern (Update-Anomalie)

Die Ursachen für das Vorkommen von Redundanz und der Speicheranomalien in der Relation *Mitarbeiter* sind folgende: In der Relation *Mitarbeiter* fällt auf, dass nicht klar zwischen der Zugehörigkeit von Attributen zu bestimmten Objekten der realen Welt unterschieden wird. In der Relation befinden sich Attribute, die zu unterschiedlichen Entity-Typen gehören und aufgrund einer Beziehung der Entity-Typen zueinander alle in die gleiche Relation *Mitarbeiter* aufgenommen wurden.

Die Attribute *Mit-Nr*, *Name* und *Beruf* gehören zum Objekt *Mitarbeiter*, die Attribute *Abteilung* und *Standort* zum Objekt *Abteilung*. Das Attribut *Abteilung* in der Relation *Mitarbeiter* fungiert eigentlich als Fremdschlüssel für die Herstellung der Beziehung zwischen den beiden Objekten *Mitarbeiter* und *Abteilung*.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Die Identifikation der funktionalen Abhängigkeiten in den Tabellen führt zu folgendem Ergebnis:

Mit-Nr → Name
Mit-Nr → Beruf
Mit-Nr → **Abteilung**
Abteilung → Standort

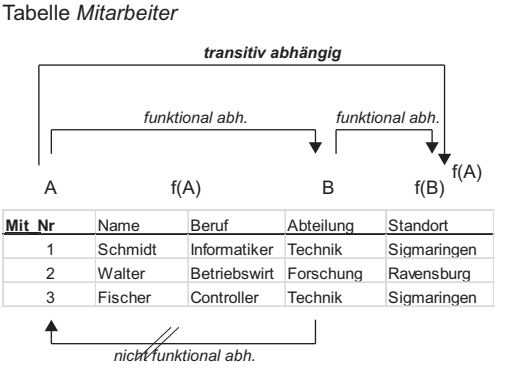
In Anwendung der Axiome von Armstrong bei funktionalen Abhängigkeiten folgt wegen der **Transitivitätseigenschaft** hieraus folgender Sachverhalt:

Mit-Nr → Standort

Festzustellen ist, dass das Attribut *Mit-Nr* nicht funktional vom Attribut *Abteilung* abhängig ist. Man spricht in diesem Fall von einer transitiven funktionalen Abhängigkeit.

In einer Relation *R(S,A,B)* ist das Attribut *B* vom Schlüssel *S* **transitiv abhängig**, wenn *A* von *S* funktional abhängig ist, *S* jedoch nicht von *A*, und *B* von *A* funktional abhängig ist.

Im vorliegenden Beispiel der Tabelle *Mitarbeiter* sind damit die in Abbildung 3.21 dargestellten funktionalen Abhängigkeiten gegeben:



Mit_Nr	Name	Beruf	Abteilung	Standort
1	Schmidt	Informatiker	Technik	Sigmaringen
2	Walter	Betriebswirt	Forschung	Ravensburg
3	Fischer	Controller	Technik	Sigmaringen

Abb. 3.21: Funktionale Abhängigkeiten in der Tabelle Mitarbeiter

Dritte Normalform (3NF): Eine Relation *R* ist in **dritter Normalform**, falls sie in 2NF ist und **keine transitiven Abhängigkeiten** von Nichtschlüsselattributen zu Primärschlüsselattributen vorliegen.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Die Auflösung bzw. Zerlegung einer 2NF-Relation in die Eigenschaft einer dritten Normalform erfolgt über das in Abbildung 3.22 visualisierte Normalisierungsschema.

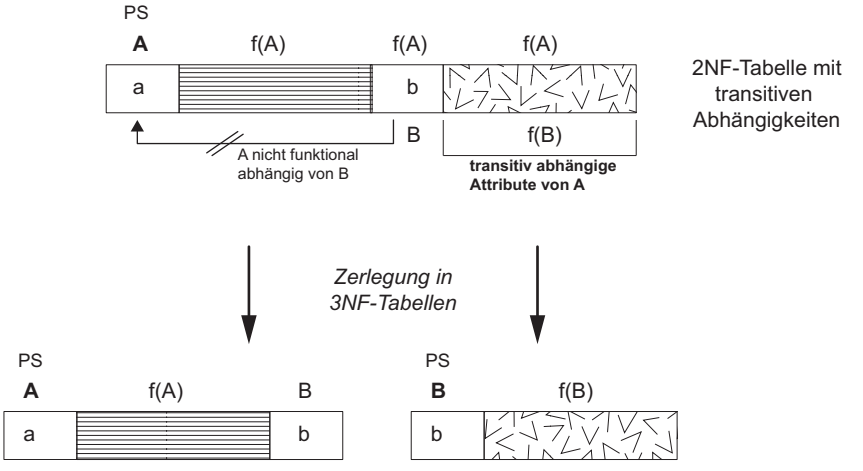


Abb. 3.22: Zerlegungsprinzip nach 3NF

Beispiel:
Nach dem Zerlegungsschema für die Herstellung der dritten Normalform müssen die transitiv abhängigen Attribute, also hier das Attribut *Standort*, herausgelöst und in einer neuen Tabelle *Abteilung* redundanzfrei eingebracht werden. Das Ergebnis zeigt Abbildung 3.23.

Tabelle *Mitarbeiter*

PS

A f(A) B

Mit_Nr	Name	Beruf	Abteilung
1	Schmidt	Informatiker	Technik
2	Walter	Betriebswirt	Forschung
3	Fischer	Controller	Technik

Tabelle *Abteilung*

PS

B f(B)

Abteilung	Standort
Technik	Sigmaringen
Forschung	Ravensburg

Abb. 3.23: Ergebnis der Zerlegung in dritte Normalform

Die erzeugten Tabellen *Abteilung* und *Mitarbeiter* erfüllen nun die dritte Normalform und es zeigen sich keine redundanten Werte mehr. Speicheranomalien sind in diesen konkreten Tabellen ebenfalls nicht mehr gegeben.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Anmerkung zum Auftreten von weiteren Speicheranomalien: in seltenen Fällen können auch bei Relationen in 3NF noch Anomalien auftreten, beispielsweise falls die Relation nur Schlüsselattribute beinhaltet und der Primärschlüssel sich aus einer Teilmenge dieser Schlüsselattribute zusammensetzt. Eine weiterführende Normalform stellt die sogenannte **Boyce-Codd'sche-Normalform** dar, die diese Art von Speicheranomalien auch bei 3NF-Relationen eliminiert.

Literaturverzeichnis

Codd, E. F. (1970): A relational model for large shared datababanks. In: Communication of ACM, Vol. 13, No. 6, S. 377–387.

Schlageter, Gunter; Stucky, Wolffried (1983): Datenbanksysteme: Konzepte und Modelle; mit einigen Tabellen und zahlreichen Beispielen. 2., neubearbeitete und erweiterte Auflage. Stuttgart: Teubner (Teubner-Studienbücher Informatik, 37).